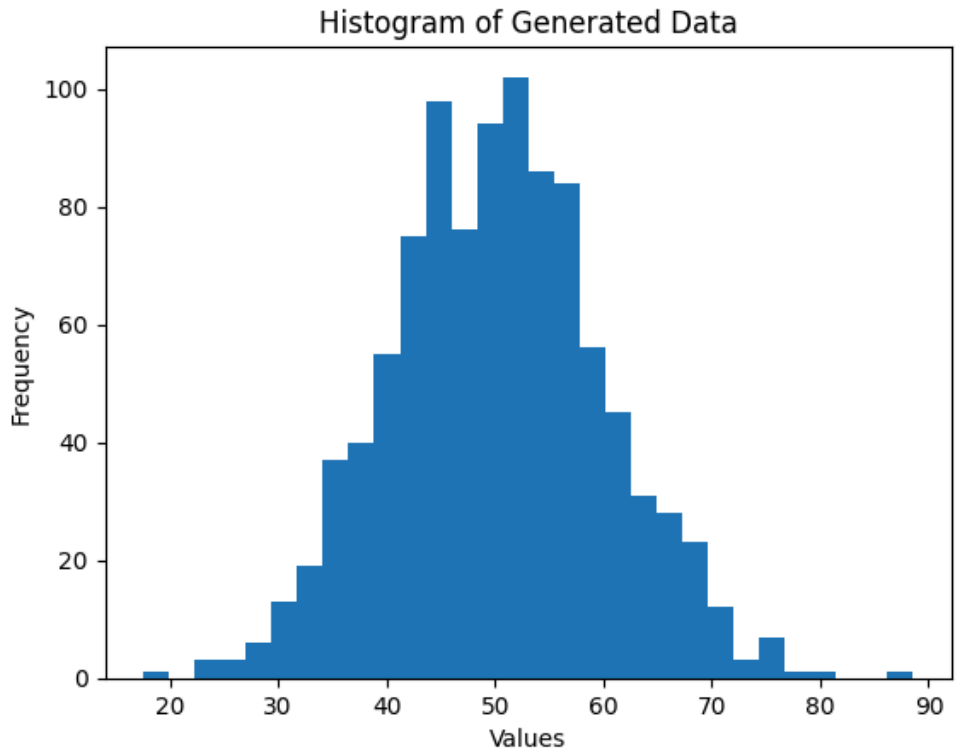
```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Generate dataset
np.random.seed(42)
data = np.random.normal(loc=50, scale=10, size=1000)

# Step 2: Manual calculations
mean_manual = sum(data) / len(data)

variance_manual = sum((x - mean_manual) ** 2 for x in data) / len(data)

std_manual = variance_manual ** 0.5

print("Mean:", mean_manual)
print("Variance:", variance_manual)
print("Standard Deviation:", std_manual)

# Step 3: Histogram
plt.hist(data, bins=30)
plt.title("Histogram of Generated Data")
plt.xlabel("Values")
plt.ylabel("Frequency")
plt.show()
```

```
Mean: 50.19332055822327
Variance: 95.79049897315171
Standard Deviation: 9.787262077473542
```




```
# Manual Z-score
z_scores = [(x - mean_manual) / std_manual for x in data]

# Verify
z_mean = sum(z_scores) / len(z_scores)
z_variance = sum((z - z_mean) ** 2 for z in z_scores) / len(z_scores)
z_std = z_variance ** 0.5
```

```
print("Z-score Mean:", z_mean)
print("Z-score Std Dev:", z_std)
```

```
Z-score Mean: -1.5523138330308938e-15
Z-score Std Dev: 0.9999999999999999
```

```
marks = [45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95, 100, 30, 20]

# Mean
mean_marks = sum(marks) / len(marks)

# Median
sorted_marks = sorted(marks)
n = len(marks)
if n % 2 == 0:
    median_marks = (sorted_marks[n//2 - 1] + sorted_marks[n//2]) / 2
else:
    median_marks = sorted_marks[n//2]

# Variance
variance_marks = sum((x - mean_marks) ** 2 for x in marks) / len(marks)

# Std Dev
std_marks = variance_marks ** 0.5

print("Mean:", mean_marks)
print("Median:", median_marks)
print("Variance:", variance_marks)
print("Std Dev:", std_marks)

# Z-score outliers
outliers = []
for x in marks:
    z = (x - mean_marks) / std_marks
    if abs(z) > 2:
        outliers.append(x)

print("Outliers:", outliers)
```

```
Mean: 65.71428571428571
Median: 67.5
Variance: 535.2040816326531
Std Dev: 23.134478201002356
Outliers: []
```

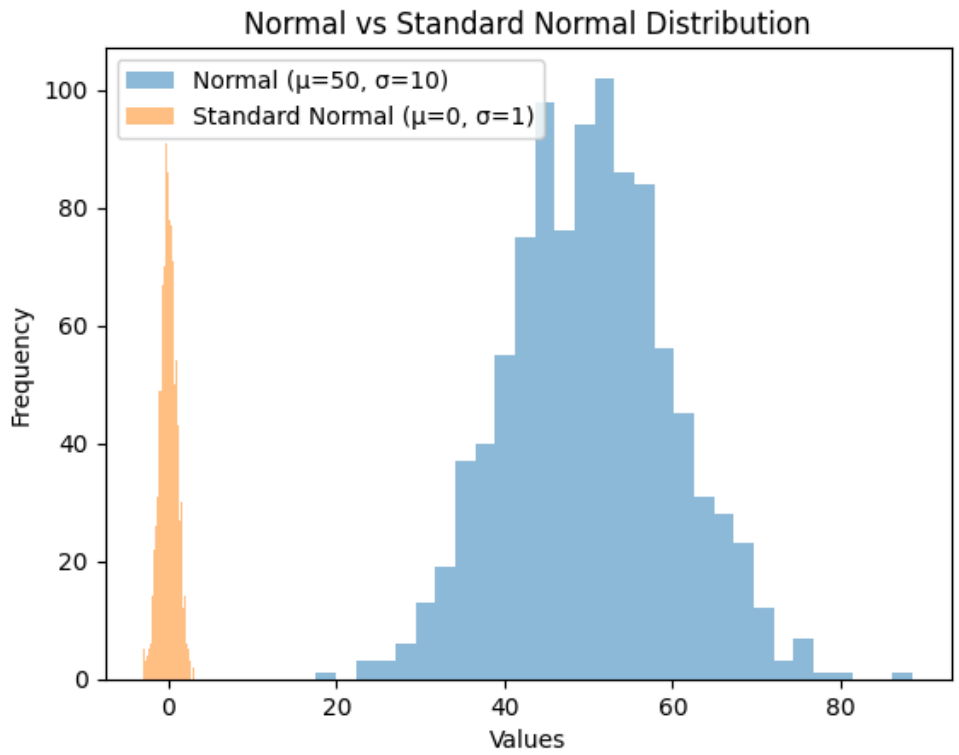
```
mu = 50
sample_mean = mean_manual
sigma = std_manual
n = len(data)

# Z-statistic
z = (sample_mean - mu) / (sigma / (n ** 0.5))

print("Z-statistic:", z)

# Critical value for alpha = 0.05 (two-tailed)
z_critical = 1.96

if abs(z) > z_critical:
    print("Reject H0")
else:
    print("Fail to reject H0")
```

```
Z-statistic: 0.6246213472997505
Fail to reject H0
```

```
num_simulations = 1000
false_positives = 0

for _ in range(num_simulations):
    sample = np.random.normal(loc=50, scale=10, size=100)

    sample_mean = sum(sample) / len(sample)
    sample_std = (sum((x - sample_mean)**2 for x in sample) / len(sample)) ** 0.5

    z = (sample_mean - 50) / (sample_std / (len(sample) ** 0.5))

    if abs(z) > 1.96:
        false_positives += 1

false_positive_rate = false_positives / num_simulations

print("False Positive Rate:", false_positive_rate)
```

```
False Positive Rate: 0.063
```

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# Normal Distribution (μ=50, σ=10)
normal_data = np.random.normal(loc=50, scale=10, size=1000)

# Standard Normal Distribution (μ=0, σ=1)
standard_data = np.random.normal(loc=0, scale=1, size=1000)

# Plot
plt.hist(normal_data, bins=30, alpha=0.5, label='Normal (μ=50, σ=10)')
plt.hist(standard_data, bins=30, alpha=0.5, label='Standard Normal (μ=0, σ=1)')
plt.legend()
plt.title("Normal vs Standard Normal Distribution")
plt.xlabel("Values")
plt.ylabel("Frequency")
plt.show()
```




```
# Generate two groups
group1 = np.random.normal(loc=50, scale=10, size=100)
group2 = np.random.normal(loc=55, scale=10, size=100)

# Mean calculation
mean1 = sum(group1) / len(group1)
mean2 = sum(group2) / len(group2)

# Difference
difference = mean2 - mean1

print("Mean1:", mean1)
print("Mean2:", mean2)
print("Difference:", difference)
```

```
Mean1: 48.88395574738524
Mean2: 54.79883293739489
Difference: 5.914877190009655
```

```
data = [10, 20, 30, 40, 50]

# Mean
mean = sum(data) / len(data)

# Standard deviation
variance = sum((x - mean) ** 2 for x in data) / len(data)
std = variance ** 0.5

# Z-scores
z_scores = [z_score(x, mean, std) for x in data]

print("Mean:", mean)
print("Standard Deviation:", std)
print("Z-scores:", z_scores)
```

```
Mean: 30.0
Standard Deviation: 14.142135623730951
Z-scores: [-1.414213562373095, -0.7071067811865475, 0.0, 0.7071067811865475, 1.41421356237309
```

```
import numpy as np

# Generate data
data = np.random.normal(50, 10, 100)

# Mean and std
mean = np.mean(data)
std = np.std(data)

# Z-score for first value
z = (data[0] - mean) / std

print("Mean:", mean)
print("Std Dev:", std)
print("Z-score:", z)

# Hypothesis test
mu = 50
n = len(data)

z_stat = (mean - mu) / (std / (n ** 0.5))

if abs(z_stat) > 1.96:
    print("Reject H0")
else:
    print("Fail to reject H0")
```

```
Mean: 49.98660441049134
```